

計算機科学基礎実験 レポート 3

氏名：加藤悠菜 / 学籍番号：6325040

2026 年 6 月 2 日

1 課題 1: 数値微分

1.1 導関数の定義からの微分係数の計算

```
let diff_forward f x =  
(f(x +. h) -. f x) /. h  
;;
```

1.2 中心差分による高精度化

```
let diff_central f x =  
(f(x +. h) -. f(x -. h)) /. (2.0 *. h)  
;;
```

1.3 ニュートン法の仕組みの調査

ニュートン法は、方程式 $g(x) = 0$ の解を、曲線の接線を利用して反復的に近似するアルゴリズムである。
与えられた関数 $f(x)$ の極値が知りたいので

$$f'(x) = 0 \quad (1)$$

という方程式の解を求める。

近似値を x_0 とする。このとき、点 $(x_0, f'(x_0))$ における導関数 $f'(x)$ の接線の方程式を考える。接線の傾きは $f'(x)$ をさらに微分した 2 階微分 $f''(x_0)$ となるため、接線の式は以下のように表される。

$$y = f''(x_0)(x - x_0) + f'(x_0) \quad (2)$$

「現在の点における接線は、真の曲線に近い振る舞いをする」という仮定に基づき、この「接線が x 軸と交わる点」を、次の近似値 x_{next} とみなす。

式 (2) に $y = 0$ および $x = x_{next}$ を代入すると、

$$-f'(x_0) = f''(x_0)(x_{next} - x_0) \quad (3)$$

この式を、次の座標である x_{next} について解くと、

$$x_{next} = x_0 - \frac{f'(x_0)}{f''(x_0)} \quad (4)$$

これを繰り返すことで、近似値は極値 ($f'(x) = 0$) へ向かって収束していく。

1.4 再帰による極値の探索

```
let rec ext f x0 =  
let df = diff_central f x0 in  
  
if abs_float df < eps then  
  (x0, f x0)  
else  
  let f_prime x = diff_central f x in  
  let ddf = diff_central f_prime x0 in  
  
  let x_next = x0 -. (df /. ddf) in  
  ext f x_next  
;;
```

1.5 精度と速度に関する考察

刻み幅 h 、初期値 x_0 が計算の精度や速度に与える影響について、指定の観点から考察を行う。

1.5.1 精度

数値微分には主に「前方差分」と「中心差分」の2種類がある。前方差分が $\frac{f(x+h)-f(x)}{h}$ という片側の情報だけをを使うのに対し、中心差分は $\frac{f(x+h)-f(x-h)}{2h}$ という x の両側の情報を使用している。前方差分は「現在の点と少し先の点を結んだ直線の傾き」を求めるため、グラフが曲がっている場合、どうしても本来の接線の傾きから大きくズレてしまう。一方、中心差分は「現在の点を真ん中に挟んだ2点」を結ぶため、グラフの曲がり具合の影響が左右でうまく相殺される。

また、前方差分の誤差が h の1乗に比例するのに対し、中心差分の誤差は h の2乗に比例することが知られている。そのため、中心差分のほうが少ない回数で、本来の微分値に近い傾きを得ることができる。微分をコンピュータで計算する際、刻み幅 h の大きさをどう設定するかは非常に重要である。

- h を大きくしすぎた場合 (例: $h = 0.5$ など):

コンピュータの計算自体は正確に行われるが、誤差が大きくなる。つまり、グラフのカーブを無視して雑に直線で結んでしまうため、微分値としての精度が低下する。

- h を小さくしすぎた場合 (例: $h = 10^{-16}$ など):

数式上は真の微分に近づくはずだが、「丸め誤差」が発生する。非常に近い値どうしの引き算を行うことで有効数字が失われ、計算結果が 0.0 に潰れたり、異常な値に化けたりして計算が破綻する。

本実験で用いた $h = 0.0001$ はバランスの良い適切な値であったと思う。

1.5.2 速度と収束性

ニュートン法は、現在の点での接線を利用して次の近似値を探すアルゴリズムである。最大の強みは、極値の近くにおける「2 次収束」という圧倒的な収束速度にある。これは、ループを 1 回まわすごとに有効数字が倍々で増えていくという性質である。そのため、滑らかな 2 次関数に対してはわずか 1~2 回の再帰ループで収束する特徴を持つ。

ニュートン法は非常に高速な反面、スタート地点である「初期値 x_0 」の選び方に結果や速度が強く依存するという弱点もある。今回、私がテスト関数で使った $f(x) = -x^2 + 4x$ は全区間で綺麗に「上に凸」であり、極値が 1 つしかない単純な関数であれば、初期値の選び方によらず全く同じ速度で確実に同じ頂点へと収束する。

しかし、もし対象の関数が、山や谷がいくつもある複雑な高次関数であった場合、初期値の選び方によって以下のような影響が出る。

- 異なる極値への収束：
初期値 x_0 に一番近い凸に引っかかり、本来の極大値を見つけられない。
- 速度の低下・発散：
選んだ初期値の付近の傾きが水平に近かった場合、更新式の分母の 2 階微分との兼ね合いで、次の座標 x_{next} が異常な場所に飛び、収束までに膨大な回数がかかったり、計算不能に陥ったりする。

したがって、ニュートン法を効果的に用いるためには、あらかじめ関数の大まかな形を把握し、本来の極値にある程度近い適切な初期値 x_0 を慎重に選んでスタートさせる必要がある。

2 課題 2: 数値積分

2.1 長方形および台形の面積を求める関数の定義

2.1.1 長方形として面積を計算する関数

```
let area_rectangle f x dx_val =  
  (f x) *. dx_val  
;;
```

2.1.2 台形として面積を計算する関数

```
let area_trapezoid f x dx_val =  
  ((f x +. f (x +. dx_val)) /. 2.0) *. dx_val  
;;
```

2.2 シンプソンの公式の仕組みの調査

1. どのような線で関数を近似しているか

これまでの長方形近似や台形近似では、微小区間 $[x, x + dx]$ の「左端」と「右端」の2点しか見ていなかった。そのため、その2点の間を直線で結ぶことしかできず、関数がカーブしていると誤差が生まれてしまっていた。これに対してシンプソンの公式では、区間の左端 x 、右端 $x + dx$ に加えて、そのちょうど真ん中にある中点 $x + \frac{dx}{2}$ の計3点に着目する。数学の性質として「3点があると、それらを通る放物線が1つに決まる」というルールがある。シンプソンの公式は、この性質を利用して、グラフの曲がり具合に合わせて滑らかな「放物線」で近似しているのが最大の特徴である。

2. 1 区間分の面積を求める数式

この「3点を通る放物線」の下側の面積を計算すると、1区間分の面積 ΔS_{simp} は次のような数式で表される。

$$\Delta S_{simp} = \frac{dx}{6} \left\{ f(x) + 4f\left(x + \frac{dx}{2}\right) + f(x + dx) \right\} \quad (5)$$

3. 数式の意味

導出された式 (5) を見ると、両端の高さ ($f(x)$ と $f(x + dx)$) に比べて、真ん中の高さ ($f(x + \frac{dx}{2})$) だけが「4倍」という大きな重みを与えられている。これは区間の真ん中の高さのほうが、両端の高さよりも、その区間全体の平均的な高さを代表する情報として強く貢献しているという放物線の面積の性質を数式で表したものである。ただの直線で結ぶ台形近似に比べて、このように中点の情報を取り込んで2次曲線を使用するため、元の関数がカーブしていても隙間を埋めることができる。そのため、今回の実験のように $dx = 0.5$ という非常に大雑把な設定であっても、誤差が全く出ずに完璧に理論値通りの9.0を導き出すことができる。

2.3 シンプソンの面積を求める関数の定義

```
let area_simpson f x dx_val =  
  let mid = x +. (dx_val /. 2.0) in  
  ((f x +. 4.0 *. f mid +. f (x +. dx_val)) /. 6.0) *. dx_val  
;;
```

2.4 共通の積分関数（高階関数）の定義と実行

実装コードは以下のようである。

```
let rec integral area_func f a b =  
  if a >= b then 0.0  
  else area_func f a dx +. integral area_func f (a +. dx) b  
;;
```

今回は各公式の精度の差を明確にするためにあえて粗い幅に設定した。

```
let dx = 0.5;;
```

また、今回、テスト関数 (my_func) として以下の二次関数を用いた。

$$f(x) = -x^2 + 4x \tag{6}$$

これを踏まえて実行結果を以下に示す。

■長方形近似

```
# integral area_rectangle my_func 0.0 3.0;;  
- : float = 8.125
```

■台形近似

```
# integral area_trapezoid my_func 0.0 3.0;;  
- : float = 8.875
```

■シンプソン公式

```
# integral area_simpson my_func 0.0 3.0;;  
- : float = 9.
```

2.5 精度に関する考察

2.5.1 高階関数を用いた設計の利点

面積の計算 (`area_rectangle` など) と、再帰処理 (`integral`) を完全に分離し、高階関数として組み合わせた。この構造のメリットは以下のようなものである。

- コードの重複を防ぎ、再利用性を高める：

もし高階関数を使わずに実装しようとする、「長方形用」「台形用」「シンプソン用」にそれぞれ別々に再帰のループ関数を書く必要があり、コードの大部分がコピー&ペーストによる重複になってしまう。合計を累積していく共通の枠組みを `integral` として 1 つにまとめたことで、プログラム全体を非常にシンプルに保つことができた。

- 拡張性とメンテナンス性が高い：

新しい別の面積計算アルゴリズムを追加したくなった場合、共通関数 `integral` の中身には一切手を加える必要がない。新しく 1 区間分の面積を計算する関数を 1 つ定義して、それを `integral` の第 1 引数に放り込むだけで対応できるため、簡単にプログラムを組める。

2.5.2 3つの近似手法による精度差

テスト関数 `my_func` を区間 $[0.0, 3.0]$ で積分した際の数学的な理論値は 9.0 である。 dx の大小がそれぞれの公式に与えた影響と精度の違いについて考察する。

■ $dx = 0.5$ という粗い設定のとき 公式ごとの精度の違いが顕著に現れた。

- 長方形近似 (8.125 / 誤差：-9.72%)：
一番雑な近似であるため、関数が山型に曲がっている今回の形状では、頂点付近の面積を正確に計算できず最も大きな誤差が出た。
- 台形近似 (8.875 / 誤差：-1.39%)：
2 点を斜めの直線で結ぶため長方形より劇的に精度が上がった。しかし、今回の関数は「上に凸」の放物線なので、結んだ直線は必ず曲線の「下側」を通る。その結果、曲線との間にできるわずかな隙間の分だけ、理論値より必ず小さくなるため誤差が残った。
- シンプソン公式 (9.0 / 誤差：0.00%)：
中点を含めた 3 点を通る「放物線」で曲面を覆う。今回のテスト関数が「2 次関数」そのものであるため、公式が裏で作る近似カーブと本物の関数の形状が完全に一致した。そのため、どれほど大雑把な分割であっても理論上誤差が完全にゼロとなり、完璧に理論値の 9.0 を計算することができた。

■ $dx = 0.001$ のような非常に細かい設定のとき 最初に行った設定 ($dx = 0.001$) では、長方形 (9.0014...)、台形・シンプソン (9.0029...) となり、どの公式を使ってもほぼ同じような値に収束した。これは、分割数を極限まで細かくしていけば、どんな公式であっても、最終的にはすべて同じ面積に収束するという積分の性質が現れたためである。

これらを踏まえて以下のことがわかった。

- dx を小さくする：

どんな公式でも確実に理論値に近づくが、再帰ループの回数が何万回・何十万回と爆発的に増えるため、計算速度が落ちる。また、あまりに細かく足し算を繰り返すと、コンピュータ特有の小さな丸め誤差が累積していくリスクもある。

- dx を大きくする：

計算スピードは一瞬で終わるが、長方形や台形のような低次の公式ではグラフのカーブを拾いきれず、精度が大幅に低下する。

これらを通してシンプソン公式のような「関数の形状にマッチした優れたアルゴリズム」を選択すれば $dx = 0.5$ という計算量が少ない設定であっても、完璧な精度で安全に結果を出せるという、数値計算における「アルゴリズム選定の重要性」を深く理解することができた。